DEVOPS E ARQUITETURA CLOUD

FASE 4 | AULA 03 -

APM E INSTRUMENTAÇÃO: GANHANDO VISIBILIDADE DO CÓDIGO À PRODUÇÃO

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

APM e Instrumentação: Ganhando Visibilidade do Código à Produção

SUMÁRIO

O QUE VEM POR AÍ? 3
HANDS ON 4
SAIBA MAIS 5
MERCADO, CASES E TENDÊNCIAS 12
O QUE VOCÊ VIU NESTA AULA? 13
REFERÊNCIAS 14

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

APM e Instrumentação: Ganhando Visibilidade do Código à Produção

O QUE VEM POR AÍ?

O monitoramento de transações ponta a ponta e a observabilidade moderna constituem pilares fundamentais na operação de sistemas distribuídos e aplicações baseadas em microsserviços. À medida que as arquiteturas de software se tornam mais complexas e interdependentes, acompanhar o percurso completo de uma requisição — desde a interação do usuário até a resposta final do sistema — torna-se essencial para garantir desempenho, disponibilidade e experiência de uso. Conceitos como métricas de desempenho, traces distribuídos e correlação com indicadores de negócio permitem compreender não apenas o que está acontecendo na aplicação, mas também por que está acontecendo e com qual impacto, fornecendo subsídios para diagnósticos precisos e ações corretivas rápidas.

Exploraremos desde a coleta e análise de dados até a distinção entre falhas de infraestrutura e de código. Serão discutidos temas como latência, throughput, instrumentação manual e automática, além de exemplos de ferramentas amplamente utilizadas no mercado, como Prometheus, Grafana e Jaeger, enfatizando a importância de alinhar indicadores técnicos com resultados de negócio, demonstrando como a observabilidade pode ser usada estrategicamente para otimizar a performance, reduzir custos operacionais e aprimorar a confiabilidade de sistemas complexos.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

HANDS ON

O monitoramento ponta a ponta e a observabilidade podem ser iniciados com a instrumentação de uma aplicação simples baseada em microsserviços. Suponha uma arquitetura composta por três componentes: um frontend em React, uma API em Node.js e um serviço de banco de dados PostgreSQL. O primeiro passo é configurar o OpenTelemetry Collector para centralizar as informações. Em seguida, instalam-se os SDKs do OpenTelemetry em cada serviço, habilitando a coleta de métricas (como latência, throughput e taxa de erro), logs estruturados e traces distribuídos. No backend Node.js, por exemplo, adiciona-se o pacote @opentelemetry/sdk-node e o @opentelemetry/instrumentation-http, permitindo rastrear automaticamente todas as requisições HTTP. Após isso, os dados coletados são exportados para o Jaeger (para visualização dos traces) e o Prometheus + Grafana (para análise de métricas e dashboards interativos).

Com essa configuração, cada requisição feita pelo frontend passa a gerar um traceID único, visível no Jaeger, mostrando o tempo gasto em cada etapa — frontend, API e banco de dados —, enquanto o Grafana exibe gráficos de latência média, número de requisições por segundo e taxa de sucesso. Esse cenário permite simular situações reais, como uma consulta SQL lenta ou uma falha de autenticação, e observar em tempo real como essas anomalias aparecem nos painéis e nos traces, consolidando os conceitos teóricos de observabilidade e demonstrando, na prática, como a correlação entre métricas, logs e traces possibilita diagnóstico rápido, melhoria de desempenho e maior confiabilidade operacional.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

APM e Instrumentação: Ganhando Visibilidade do Código à Produção

SAIBA MAIS

O avanço das arquiteturas de software modernas, caracterizadas por sistemas distribuídos, microsserviços, ambientes em nuvem e aplicações altamente escaláveis, trouxe novos desafios para o monitoramento e a manutenção de confiabilidade operacional. Nesse contexto, o monitoramento de transações ponta a ponta, associado ao uso de métricas de desempenho, traces distribuídos e instrumentação avançada, tornou-se uma prática essencial para compreender o comportamento sistêmico de uma aplicação e garantir que a experiência do usuário final mantenha níveis aceitáveis de qualidade. A observabilidade, nesse cenário, vai além da simples coleta de dados de execução. Ela consiste na capacidade de compreender o que está acontecendo dentro de um sistema complexo a partir das informações geradas por ele — sejam métricas, logs ou traces.

O conceito de transação ponta a ponta emerge como um dos pilares dessa observabilidade moderna. Ele propõe uma visão holística da jornada de uma requisição, acompanhando-a desde o ponto de origem — tipicamente o usuário — até sua conclusão no backend, passando por todas as camadas intermediárias, como APIs, serviços internos, filas, bancos de dados e redes. Esse acompanhamento detalhado não apenas permite identificar falhas, mas também compreender as causas fundamentais que as geraram.

Iremos abordar o funcionamento, os componentes e a importância do monitoramento de transações ponta a ponta e seus elementos correlatos, abordando desde o papel das métricas de desempenho e dos traces distribuídos até a correlação de dados técnicos com indicadores de negócio. Também serão discutidas as abordagens de instrumentação manual e automática, que constituem as bases práticas para a coleta de dados de observabilidade em sistemas modernos.

Transação Ponta a Ponta: Conceito e Relevância

O monitoramento de transações ponta a ponta (*End-to-End Transaction Monitoring*) é o processo que visa acompanhar uma requisição em todo o seu percurso dentro do sistema, desde o momento em que é iniciada pelo usuário até o instante em que a resposta é devolvida. Essa abordagem permite compreender o comportamento integral da aplicação sob a perspectiva do fluxo de execução. Em sistemas

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

monolíticos, esse acompanhamento era relativamente simples, pois todas as operações ocorriam em um único ambiente de execução. Entretanto, com o surgimento dos microsserviços, containers e arquiteturas orientadas a eventos, o trajeto de uma requisição tornou-se fragmentado e distribuído entre múltiplos componentes.

Em um exemplo clássico de comércio eletrônico, uma requisição de compra pode iniciar no navegador do usuário, seguir para um serviço de frontend hospedado em nuvem, que chama uma API de catálogo, que por sua vez interage com um microsserviço de estoque, um serviço de pagamento e um banco de dados. Cada uma dessas etapas pode estar hospedada em ambientes distintos, em regiões geográficas diferentes e sob tecnologias heterogêneas. O monitoramento ponta a ponta, nesse contexto, tem como objetivo reconstruir o caminho percorrido pela requisição, identificar a contribuição de cada componente no tempo total de resposta e localizar onde ocorrem gargalos, falhas ou perdas de desempenho.

A grande relevância dessa técnica está na sua capacidade de unir a perspectiva técnica e a perspectiva de experiência do usuário. Para o usuário final, a percepção de desempenho é unificada — ele apenas percebe se a aplicação está rápida ou lenta. Contudo, para a equipe técnica, identificar qual parte da cadeia causa a lentidão exige visibilidade integral. O monitoramento ponta a ponta fornece exatamente essa visão, permitindo mapear a requisição do início ao fim, medir tempos de processamento e detectar anomalias em qualquer camada.

Ferramentas de monitoramento modernas, como o Jaeger e o OpenTelemetry, são projetadas para capturar e visualizar esse tipo de dado, fornecendo diagramas temporais que representam cada etapa da transação. Esse tipo de visualização é fundamental para diagnósticos rápidos e eficientes, pois reduz significativamente o tempo médio de detecção (MTTD) e o tempo médio de resolução (MTTR) de incidentes.

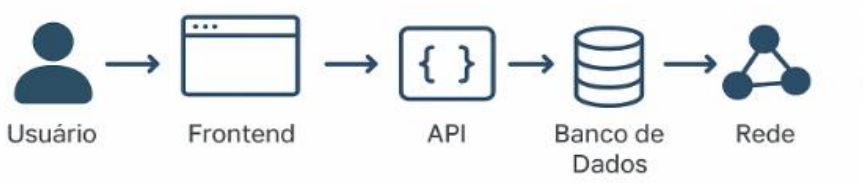


Figura 1 – Transação Ponta a Ponta

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

APM e Instrumentação: Ganhando Visibilidade do Código à Produção

Fonte: Elaborado pelo autor (2025)

Métricas de Desempenho: Indicadores Fundamentais de Eficiência

A observabilidade de um sistema baseia-se em três pilares principais: métricas, logs e traces. As métricas de desempenho são particularmente importantes porque traduzem o comportamento operacional da aplicação em valores quantitativos que podem ser analisados, comparados e correlacionados ao longo do tempo. Entre as métricas mais relevantes estão a latência, o throughput, a taxa de erro e o consumo de recursos computacionais.

A latência representa o tempo total que uma requisição leva para ser processada, desde sua origem até a entrega da resposta. É um dos indicadores mais críticos, pois impacta diretamente a experiência do usuário. Em serviços sensíveis à resposta rápida, como mecanismos de busca ou sistemas financeiros, até mesmo variações de milissegundos podem ter impacto perceptível. O monitoramento da latência é realizado tanto de forma global (medindo o tempo de resposta médio de toda a aplicação) quanto de forma granular (avaliando o tempo de execução de cada serviço ou endpoint).

O throughput, ou taxa de processamento, mede a quantidade de requisições processadas por unidade de tempo. Essa métrica é fundamental para avaliar a escalabilidade e a capacidade de carga do sistema. Um throughput estável com baixa latência é sinal de boa saúde operacional, enquanto throughput reduzido combinado com alta latência geralmente indica saturação de recursos ou gargalos internos.

A taxa de erro mensura o percentual de requisições que resultam em falha. Pode incluir falhas de rede, exceções de aplicação, respostas com códigos HTTP de erro ou falhas em chamadas de serviços externos. Essa métrica é essencial para determinar a confiabilidade do sistema e, quando analisada em conjunto com a latência, pode indicar correlações significativas — por exemplo, aumento de latência seguido de aumento de falhas pode sugerir sobrecarga de processamento.

O uso de recursos (CPU, memória, disco, rede) complementa o conjunto de métricas, fornecendo o contexto físico em que a aplicação está operando. Uma aplicação pode apresentar tempos de resposta altos simplesmente por falta de recursos disponíveis. Por isso, a análise de desempenho deve sempre correlacionar

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

métricas de aplicação e métricas de infraestrutura. Ferramentas como Prometheus e Grafana são amplamente utilizadas para coletar e visualizar essas métricas, oferecendo painéis dinâmicos que auxiliam na detecção de anomalias.

Traces Distribuídos: Visibilidade em Ambientes de Microsserviços

Com o surgimento dos microsserviços, a simples coleta de métricas deixou de ser suficiente para compreender o comportamento sistêmico. Os traces distribuídos surgem como solução para rastrear a trajetória completa de uma requisição à medida que ela atravessa diversos serviços. Um trace distribuído é uma sequência de eventos relacionados a uma mesma transação, identificada por um traceID, que acompanha a requisição ao longo de todo o seu percurso. Dentro de cada trace, cada operação é registrada como um span, que possui um identificador próprio (spanID) e uma relação hierárquica com outros spans.

Essa estrutura hierárquica forma uma árvore de execução, onde o primeiro span (chamado *root span*) representa o início da requisição e os spans filhos representam as operações subsequentes. Cada span contém informações como tempo de início, tempo de término, metadados da operação e eventuais erros. Essa abordagem permite identificar qual parte da transação consumiu mais tempo, quais chamadas externas ocorreram e quais componentes apresentaram falhas.

Em um ambiente distribuído, a propagação do contexto do trace é essencial. Isso é feito por meio da passagem do traceID em cabeçalhos de requisições HTTP, mensagens gRPC ou filas de mensagens. Assim, cada serviço envolvido na requisição sabe que pertence a um mesmo trace global e pode contribuir com seus spans.

Ferramentas de tracing como Jaeger, Zipkin e AWS X-Ray são amplamente utilizadas em conjunto com o OpenTelemetry, que é o padrão aberto e unificado de instrumentação, permitindo que aplicações gerem traces compatíveis com múltiplas plataformas de observabilidade. Essa interoperabilidade é crucial em ecossistemas modernos, onde coexistem diferentes linguagens, frameworks e fornecedores de infraestrutura.

O tracing distribuído fornece o elo que faltava entre métricas e logs. Enquanto métricas oferecem visão estatística e logs fornecem detalhes textuais, os traces

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

mostram o caminho real percorrido pela requisição, possibilitando correlação temporal e causal entre eventos.

Contexto de Usuário e Negócio: Observabilidade Orientada a Impacto

O monitoramento técnico, por si só, não é suficiente para garantir valor ao negócio. É fundamental correlacionar as métricas técnicas com o impacto real sobre os usuários e os resultados da empresa. Essa prática é conhecida como observabilidade orientada a contexto de negócio. Ela busca compreender como as condições técnicas de uma aplicação — como aumento de latência ou taxa de erro — afetam indicadores como taxa de conversão, engajamento, receita ou satisfação do cliente.

Um exemplo ilustrativo ocorre em plataformas de e-commerce. Uma degradação de desempenho na etapa de checkout pode não causar falhas aparentes, mas pode reduzir significativamente o número de transações concluídas. Nesse caso, a métrica técnica (latência elevada) tem um reflexo direto em uma métrica de negócio (queda de conversões). A correlação desses dados permite não apenas detectar o problema, mas quantificar seu impacto econômico.

Ferramentas modernas de observabilidade permitem integrar dados de negócio aos painéis técnicos, fornecendo uma visão unificada. Essa integração torna possível a criação de alertas mais inteligentes, que consideram o impacto real. Por exemplo, um alerta pode ser disparado apenas quando a latência afeta um serviço crítico de faturamento, e não qualquer endpoint secundário. Essa abordagem orientada a contexto aumenta a eficiência operacional e evita sobrecarga de alertas irrelevantes.

Detecção e Diagnóstico de Problemas

A capacidade de detectar e diagnosticar problemas rapidamente é um dos principais objetivos da observabilidade. Com a coleta contínua de dados, é possível aplicar algoritmos de detecção automática de anomalias, que identificam variações inesperadas nas métricas. Sistemas mais avançados utilizam aprendizado de máquina para prever comportamentos anômalos antes que se tornem incidentes críticos.

O diagnóstico eficaz depende da correlação entre múltiplas fontes de dados. Uma simples elevação da latência pode ter causas diversas: falhas de rede, saturação de CPU, aumento de concorrência, consultas lentas ao banco de dados ou até

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

APM e Instrumentação: Ganhando Visibilidade do Código à Produção

problemas de código. A análise combinada de métricas, logs e traces é o que permite diferenciar a natureza da falha. Por exemplo, se os traces mostram aumento de tempo de execução em um único span de banco de dados, o problema provavelmente está em uma query específica. Já se todos os serviços apresentam lentidão simultânea, o diagnóstico aponta para falha na infraestrutura compartilhada.

O uso de tracing distribuído reduz substancialmente o tempo de diagnóstico, pois fornece uma representação visual da cadeia de execução. Esse tipo de visibilidade é indispensável em arquiteturas complexas, onde uma falha em um único serviço pode se propagar e afetar toda a aplicação.

Instrumentação Manual e Automática

Para que todos os dados de observabilidade possam ser coletados, é necessária a instrumentação do sistema. A instrumentação manual consiste na inserção explícita de código que registra métricas, spans ou logs em pontos específicos da aplicação. Essa abordagem oferece controle total ao desenvolvedor, permitindo definir com precisão o que medir e como medir. Por exemplo, é possível instrumentar manualmente o tempo de execução de uma função crítica, a latência de uma consulta SQL ou o número de requisições rejeitadas por limitação de taxa. Os principais benefícios são a granularidade e a personalização, mas o custo é o aumento da complexidade e do esforço de manutenção.

A instrumentação automática, por outro lado, utiliza agentes, bibliotecas ou middleware que interceptam chamadas comuns (como HTTP, JDBC, gRPC) e coletam dados sem necessidade de alterar o código da aplicação. Essa abordagem acelera a implementação e garante cobertura ampla e consistente. Um exemplo é o uso de agentes Java que se integram a JVM e capturam automaticamente métricas de desempenho de frameworks populares. Embora menos flexível, a instrumentação automática é ideal para padronizar a coleta de dados em ambientes grandes e heterogêneos.

Em muitos casos, a melhor estratégia é a combinação de ambas as abordagens. A instrumentação automática fornece uma base ampla e rápida de monitoramento, enquanto a instrumentação manual adiciona profundidade e contexto em pontos críticos de negócio.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

APM e Instrumentação: Ganhando Visibilidade do Código à Produção

Conclusão

O monitoramento de transações ponta a ponta e o tracing distribuído representam o estado da arte da observabilidade moderna. Eles permitem não apenas identificar falhas técnicas, mas compreender o comportamento completo de sistemas distribuídos sob a perspectiva do usuário e do negócio. A adoção de métricas bem definidas, a correlação entre dados técnicos e impacto real, e o uso de instrumentação adequada constituem elementos indispensáveis para a operação confiável de aplicações em larga escala.

A observabilidade, portanto, deve ser entendida como um processo contínuo de aprendizado sobre o próprio sistema. Ao transformar dados em entendimento e entendimento em ação, ela viabiliza não apenas a detecção de falhas, mas a evolução constante da qualidade de software e da experiência do usuário.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

APM e Instrumentação: Ganhando Visibilidade do Código à Produção

MERCADO, CASES E TENDÊNCIAS

Understanding OpenTelemetry Architecture and Components - Link.

The Hidden Cost of Observability: Breaking Free from the Vendor Lock-In Tax - Link.

Plataformas Unificadas e Consolidação de Ferramentas - Link.

Beyond Monitoring -The Evolution of Observability in Modern Systems - Link.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

APM e Instrumentação: Ganhando Visibilidade do Código à Produção

O QUE VOCÊ VIU NESTA AULA?

Abordamos a evolução das práticas de observabilidade em sistemas modernos, destacando a importância do monitoramento de transações ponta a ponta como pilar fundamental para compreender o comportamento de aplicações distribuídas. Com o avanço de arquiteturas baseadas em microsserviços, nuvem e eventos, o acompanhamento completo de uma requisição — do usuário ao backend — tornou-se essencial para identificar gargalos, falhas e impactos na experiência do usuário. Nesse contexto, as métricas de desempenho (como latência, throughput e taxa de erro) e os traces distribuídos são apresentados como instrumentos-chave para mensurar eficiência, localizar problemas e entender a jornada de execução das requisições em ambientes complexos.

A necessidade de correlacionar dados de observabilidade com indicadores de negócio promove uma visão orientada a impacto. Ferramentas como Jaeger, Prometheus, Grafana e OpenTelemetry são soluções integradas que facilitam a coleta, visualização e análise desses dados. Aliado a essas ferramentas, temos ainda as abordagens de instrumentação manual e automática, destacando que a combinação de ambas oferece o equilíbrio ideal entre controle detalhado e agilidade na implementação.

Assim, a observabilidade é apresentada não apenas como um meio de detectar falhas, mas como um processo contínuo de aprendizado e melhoria da qualidade e confiabilidade dos sistemas.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

APM e Instrumentação: Ganhando Visibilidade do Código à Produção

REFERÊNCIAS

BEYER, JONES, PETOFF & MURPHY. **Engenharia de Confiabilidade do Google.** São Paulo: Novatec, 2022.

BURNS, Brendan; GRANT, Brian; OPPENHEIMER, David; BREWER, Eric; VOGELS, Werner. **Borg, Omega, and Kubernetes: Lessons Learned from Three Container-Management Systems over a Decade.** *ACM Queue*, v. 14, n. 1, 2020.

HIGHTOWER, Kelsey; BURNS, Brendan; BEDA, Joe. **Kubernetes: Up and Running – Dive into the Future of Infrastructure.** 2. ed. O'Reilly Media, 2017.

NEW RELIC. **State of Observability Report 2025.** Disponível em: https://newrelic.com/pt/resources/report/observability-forecast/2025. Acesso em: 22 jan. 2026.

OPENOBSERVABILITY. **OpenTelemetry Specification.** Disponível em: https://opentelemetry.io/docs. Acesso em: 22 jan. 2026.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

APM e Instrumentação: Ganhando Visibilidade do Código à Produção

PALAVRAS-CHAVE

Monitoramento. Observabildade. Métricas. Análise.Sistemas.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com



PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com